Production-Ready Architecture, Security Model, Data Schema & Implementation Guide

ROLE	AUTHOR	RECIPIENT	VERSION & DATE
Lead Developer Assignment	**Yash Shiva**	**tech@bidveda.in**	**v1.0.0 • October 2026**

Author: Yash Shiva (Lead Developer)
Submission: tech@bidveda.in
Project: Mini RFQ (Request for Quotation) Portal
Document Version: 1.0.0
Date: October 2026

🏷 Assignment Specification & Required Structure

> ***Assignment Brief***:
> *Write a short design document (a Markdown file in the repo, a PDF or a Google Doc). We suggest writing it before you start coding, since it will guide your build. You will submit it together with the code and the video in a single submission. It should contain these sections:*

#	Section	What We Expect	Implementation Reference
1	**Requirement understanding**	The problem in your own words, the main user flows per role, what is in scope and what is out of scope.	Section 1: Requirement Understanding
2	**Assumptions and open questions**	Every assumption you made where the requirement was unclear, with a one-line reason.	Section 2: Assumptions and Open Questions
3	**Tech stack and reasoning**	Frontend, backend, database, file storage, deployment. Why each, and what you considered instead.	Section 3: Tech Stack and Reasoning
4	**Architecture and services**	The components needed (app, database, file storage, background jobs, cache, anything else) and how they talk to each other. A simple diagram is enough.	Section 4: Architecture and Services
5	**Database model**	An ER diagram or table definitions with keys, constraints and important indexes. Explain how you store line items, quote revisions, status history and money.	Section 5: Database Model
6	**API design**	List of endpoints with method, path, role allowed, and a short purpose. Include request and response examples for the 3 most important ones.	Section 6: API Design
7	**Key flows**	Step by step for: submitting a quote before the deadline, rejecting a late quote, awarding an RFQ.	Section 7: Key Flows
8	**Security and access control**	How roles are enforced, how one vendor is prevented from reading another vendor's data, how file uploads are made safe.	Section 8: Security and Access Control
9	**Test cases**	The test cases you thought of for the main flows and edge cases. Mark which ones you implemented and which ones you would add with more time.	Section 9: Test Cases
10	**Scalability and next steps**	What breaks first at 100x load, what you would change, and what you skipped on purpose.	Section 10: Scalability and Next Steps

> ***Design Evolution Note***:
> *You may update the design document while coding. If the final code differs from your design, add a short note on what changed and why. That is normal and we like to see it.*
> *➔ See Section 11: Design Evolution & Build Notes for deviations, design learnings, and runtime enhancements made during development.*

---

1. Requirement Understanding

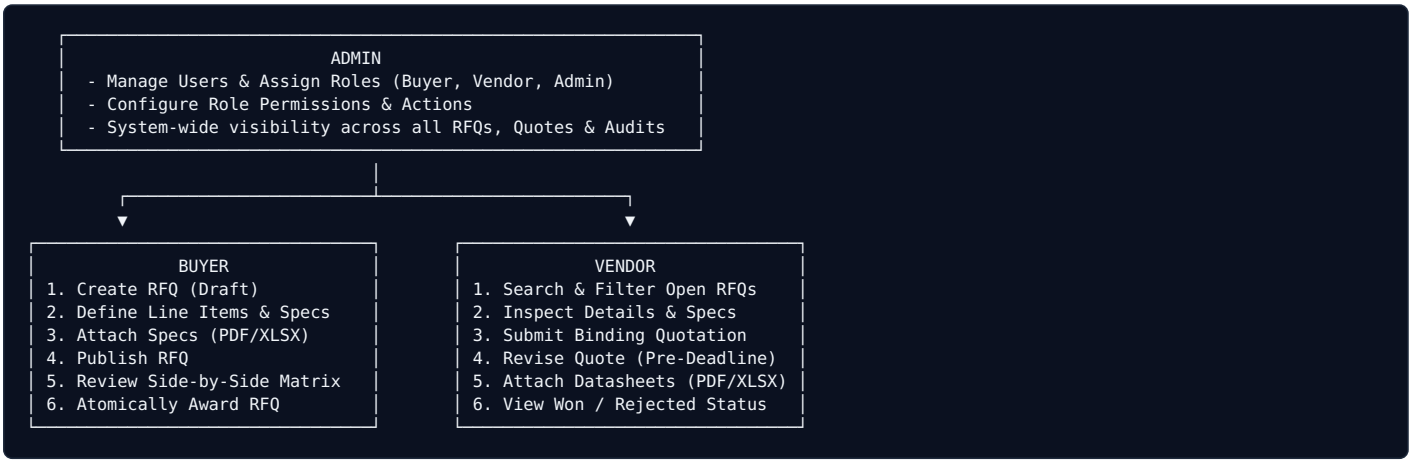
1.1 The Problem in Our Own Words

Procurement workflows in modern enterprises are often fragmented across email chains, disparate spreadsheets, and uncoordinated negotiations. This lack of structure leads to data leakage, vendor collusion, missed deadlines, and suboptimal award decisions.

The **Mini RFQ Portal** is a secure, role-based B2B procurement platform that digitizes and standardizes the entire procurement lifecycle:

1. **Buyers** publish detailed requests with multi-item line requirements, delivery locations, specifications, and strict deadlines.
2. **Vendors** discover relevant business opportunities, review requirements, submit binding itemized quotes with delivery commitments, and revise quotes prior to deadline cutoff.
3. **Sealed-Bid Integrity** ensures competing vendors cannot view rival offers during the bidding window.
4. **Post-Deadline Evaluation & Atomic Award** enables buyers to compare quotes side-by-side (sorting by price or delivery schedule) and execute an atomic award that officially declares a winner and auto-rejects competing bids.

1.2 Main User Flows per Role



A. Buyer Flow

1. **Creation**: Creates an RFQ with Title, Description, Delivery Location, Currency, Submission Deadline, and optional Target Budget.
2. **Itemization**: Adds structured line items (Item Name, Description, Quantity, Unit, Technical Specs).
3. **Documentation**: Attaches specification files or Bill of Materials (BOM) in PDF or XLSX format.
4. **Lifecycle Control**: Edits drafts, publishes to the public vendor marketplace, or cancels/deletes drafts.
5. **Comparison & Evaluation**: Accesses side-by-side quote comparison matrix, sorting by total cost or delivery speed.
6. **Award Decision**: Formally accepts one winning quote. The system atomically awards the RFQ and rejects all rival bids.

B. Vendor Flow

1. **Discovery**: Searches open published RFQs by keyword, category, delivery location, and deadline with pagination.
2. **Analysis**: Inspects line item breakdowns, target units, and downloads attached buyer specifications.
3. **Submission**: Submits a quote providing unit prices per line item, calculated totals, delivery timeline (e.g., "7 days"), and proposal notes.
4. **Revision**: Updates pricing or timeline before the deadline. Old quotes are automatically archived into an immutable revision snapshot.
5. **Deadline Cutoff**: After the deadline passes, submissions and revisions are strictly rejected by the server.

C. Administrator Flow

1. **RBAC Control**: Manages user accounts and assigns roles (`buyer` , `vendor` , `admin`).
2. **Permission Matrix**: Configures fine-grained actions per role (`rfq:create` , `rfq:read` , `quotation:create` , etc.).
3. **Platform Governance**: Oversees platform activity, audit histories, and file storage.

1.3 Scope Boundaries

- **In Scope**:
  - Role-based authentication (JWT) for Buyer, Vendor, and Admin.
  - Complete RFQ lifecycle: `DRAFT -> PUBLISHED -> CLOSED -> AWARDED` .
  - Complete Quotation lifecycle: `SUBMITTED -> SHORTLISTED -> ACCEPTED / REJECTED` .
  - Multi-item line requirements with exact financial decimal calculations.
  - Quote revisions with audit snapshots (`QuotationRevision`).
  - Hard deadline enforcement guard.
  - Side-by-side comparison matrix with ranking by price, delivery speed, and date.
  - Atomic transactional award flow with automatic competitor rejection.
  - Secure document attachments (PDF & XLSX) with Cloudinary integration and local volume fallback.
  - Search, filtering (keyword, category, location, deadline), and navigation badges.
  - Audit logging (`History` table).
- **Out of Scope**:

- Integrated payment gateways (e.g., Razorpay/Stripe Escrow payouts).
- Live currency FX auto-conversion (RFQs enforce a single base currency).
- Live audio/video supplier interviews.
- Docker containerization & automated testing test runner scripts (managed via standard npm runtime & manual API verification).

---

2. Assumptions and Open Questions

#	Question / Requirement Ambiguity	Decision / Assumption	Architectural Reason
1	Can a vendor see other vendors' quotes?	**Strictly Hidden (Sealed-Bid)**	Prevents price collusion, predatory undercut bidding, and preserves procurement integrity. Competitor quotes return `403 Forbidden`.
2	What happens to the old version when a vendor revises a quote?	**Archived in `QuotationRevision`**	Regulatory compliance requires a full audit trail. Each revision increments `revisionNo` and saves an immutable JSON snapshot.
3	How should currency be handled across multi-item RFQs?	**RFQ Base Currency**	To prevent currency volatility distortion, all quotations submitted for an RFQ inherit the RFQ's base currency (e.g., USD, INR).
4	Can a Buyer edit an RFQ after it is Published?	**Restricted / Draft Only**	Once vendors start bidding on a Published RFQ, altering quantities or specs would invalidate active bids. Major edits require closing or re-issuing.
5	Can quotes be revised after the deadline?	**Strict Zero-Tolerance Cutoff**	Server checks `deadline < UTC_NOW` before accepting any quote creation or revision. Late submissions return `400 Bad Request`.
6	What happens to competing quotes when an RFQ is awarded?	**Atomic Bulk-Rejection**	Once a quote is accepted, all remaining quotes for that RFQ transition to `REJECTED`, and the RFQ transitions to `AWARDED` in a single ACID transaction.
7	How should file uploads behave if third-party cloud credentials are unset?	**Dual Storage with Fallback**	For evaluators running the project locally, missing Cloudinary keys gracefully fall back to local disk storage (`/uploads`) so evaluation never breaks without third-party setup.

---

3. Tech Stack and Reasoning

```
┌──────────────────────────────────────────────────────────────────┐
│                        FULL-STACK TECH STACK                     │
├──────────────────────┬───────────────────────────────────────────┤
│ Frontend             │ Next.js 14+ (App Router), React, Tailwind CSS │
│ Backend API          │ Node.js, Express 5, TypeScript            │
│ Database & ORM       │ PostgreSQL 16, Prisma ORM 7 (@prisma/adapter-pg) │
│ Authentication       │ JWT (jsonwebtoken), Bcrypt password hashing │
│ File Storage         │ Cloudinary API + Local Disk Storage Fallback │
│ Runtime Environment  │ Node.js 20+, tsx watch, npm               │
│ API Verification     │ Manual REST API & Postman Verification    │
└──────────────────────┴───────────────────────────────────────────┘
```

3.1 Architectural Reasoning & Alternatives Considered

- **Frontend: Next.js (App Router) + Tailwind CSS**
  - *Why*: Next.js provides rapid SSR/client-component separation, instant routing, and high UI responsiveness. Tailwind CSS enables a rich, modern enterprise UI design with clean contrast and accessible tables.
  - *Considered Instead*: Create-React-App / SPA (lacks server-side rendering, slower initial load times, and poor metadata management).
- **Backend: Express + TypeScript (Container 3-Tier Pattern)**
  - *Why*: Node.js has an asynchronous event-driven model suited for concurrent I/O (handling sudden bursts of quote submissions near deadlines). TypeScript guarantees compile-time contract enforcement between controllers, services, and repositories.
  - *Considered Instead*: Python / Django (heavier resource footprint, slower concurrency on raw I/O, lacks unified TS type sharing).
- **Database: PostgreSQL 16 + Prisma ORM 7**
  - *Why*: Relational databases are mandatory for procurement. PostgreSQL provides ACID transactions, foreign key constraints, composite unique indexes (`[rfqId, vendorId]`), and precise `Decimal(12, 2)` monetary arithmetic. Prisma 7 ensures type-safe database queries and automated schema migrations.
  - *Considered Instead*: MongoDB (document stores lack native multi-table transactional guarantees, risking orphaned quotations or dirty reads during award flows).
- **File Storage: Dual Cloudinary + Local Disk Volume**
  - *Why*: Cloudinary provides CDN streaming and cloud delivery. However, requiring evaluators to register personal cloud keys creates evaluation

friction. Implementing a seamless fallback to `./uploads` guarantees the app runs instantly with `npm run dev`.

- *Considered Instead*: AWS S3 (requires AWS IAM keys and billing; unnecessary barrier for assignment evaluation).

---

4. Architecture and Services

The application follows an enterprise **Container-Driven 3-Tier Architecture**:

```
[ Client: Next.js Web App ]
            | HTTP / HTTPS (JSON & Multipart)
            ▼
[ API Gateway / Express Router (src/routes) ]
            |
            ├─► [ Middlewares ]: Passport JWT Auth, RBAC Authorization, Multer
            |
            ▼
   [ Controller Layer ] (Request unwrapping, validation, HTTP response formatting)
            |
            ▼
     [ Service Layer ] (Business rules, deadline checks, role logic, data derivations)
            |
            ▼
   [ Repository Layer ] (Prisma DB transactions, raw queries, ACID award operations)
            |
   ┌────────┴──────────────────────────────┐
   ▼                                       ▼
[ PostgreSQL Database ]        [ Storage: Cloudinary / Disk ]
```

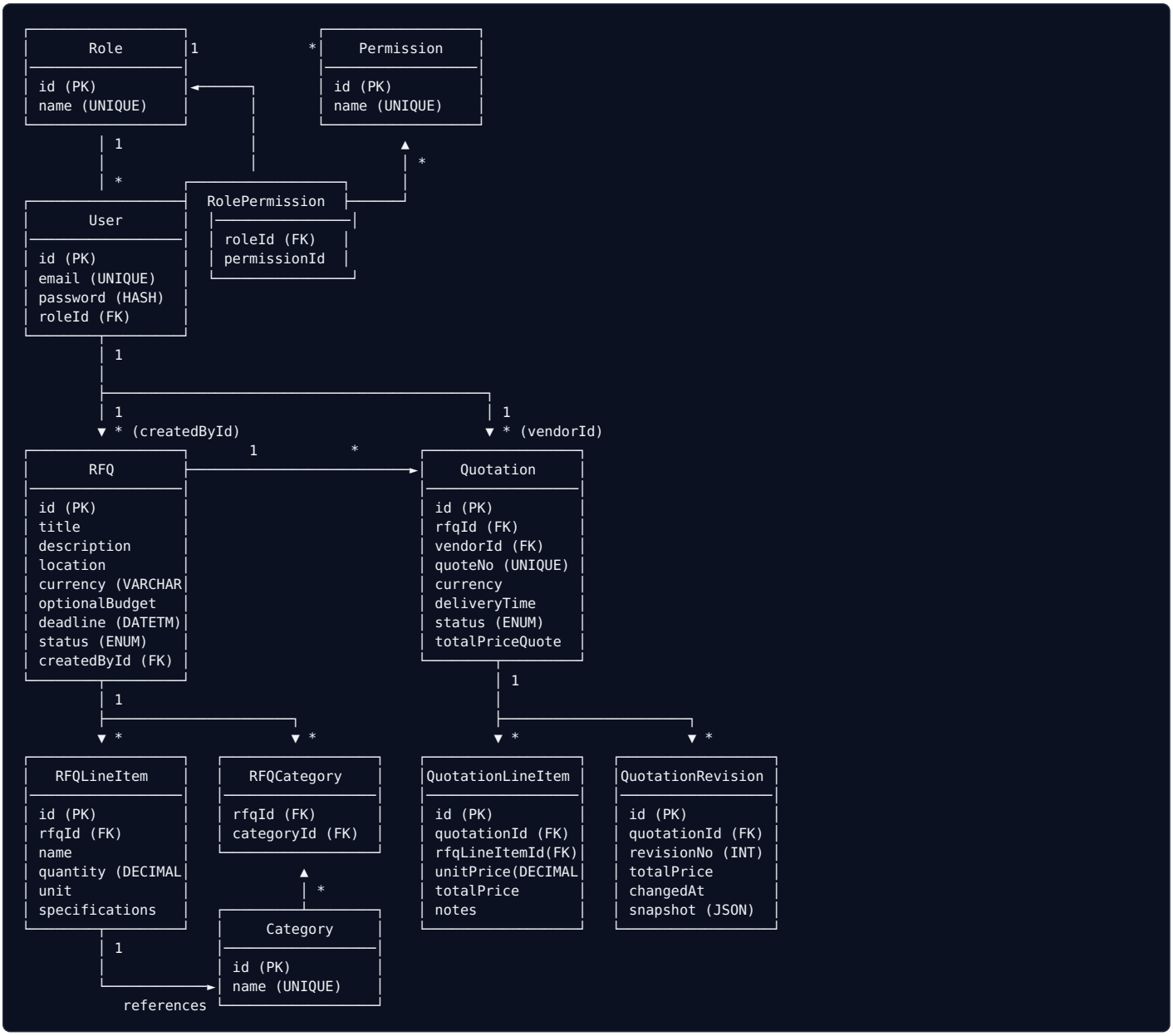
Component Breakdown

1. **Next.js Frontend Client**: Renders the RFQ creation wizard, side-by-side quote comparison table, vendor bidding drawer, file attachment managers, and admin RBAC console.
2. **Express API Server**: Exposes stateless RESTful `/v1` endpoints with centralized error handling, internationalized messages (`i18n`), and standard response envelopes.
3. **Database (PostgreSQL)**: Persists entities with foreign keys, cascaded deletions, and unique constraints.
4. **Cloudinary / Local Upload Service**: Streams incoming PDF and XLSX buffers into cloud buckets or local disk storage (`/uploads`).

---

5. Database Model

5.1 Entity-Relationship Diagram (ERD)



5.2 Key Data Integrity Rules

- **Financial Calculations**: All price and budget columns use `Decimal(12, 2)` (not floating-point `FLOAT` or `DOUBLE`) to guarantee cent-accurate calculations without rounding errors.
- **One Quote Per Vendor per RFQ**: `@@unique([rfqId, vendorId])` guarantees a vendor cannot spam multiple parallel submissions for the same RFQ; revisions must update the single active quote.
- **Item Pairing**: `@@unique([quotationId, rfqLineItemId])` ensures every quotation line item uniquely maps to an authorized RFQ line item.
- **Revision Tracking**: `@@unique([quotationId, revisionNo])` stores sequential version snapshots (`1, 2, 3...`) with the full JSON state and timestamp.
- **Audit Trail (`History`)**: Every status change (`DRAFT -> PUBLISHED`, `SUBMITTED -> ACCEPTED`) records who triggered it, when, and the change details in a dedicated `History` table.

---

6. API Design

6.1 Complete Endpoint Directory

Method	Path	Role Allowed	Purpose
`POST`	`/v1/auth/signup`	Public	Register new Buyer or Vendor
`POST`	`/v1/auth/login`	Public	Authenticate user & receive JWT token
`GET`	`/v1/user/profile`	Authenticated	Fetch current user profile & role
`GET`	`/v1/user`	Admin	List all registered users
`PUT`	`/v1/user/:id/role`	Admin	Assign role to user
`GET`	`/v1/role`	Authenticated	List roles & assigned permissions
`PUT`	`/v1/role/:id/permissions`	Admin	Configure actions/permissions for a role
`POST`	`/v1/rfq`	Buyer, Admin	Create RFQ with line items & categories
`GET`	`/v1/rfq`	Authenticated	List & filter RFQs (scoped by role)
`GET`	`/v1/rfq/:id`	Authenticated	Get RFQ details and line items
`PUT`	`/v1/rfq/:id`	Buyer, Admin	Update draft RFQ
`PATCH`	`/v1/rfq/:id/publish`	Buyer, Admin	Publish RFQ to marketplace
`GET`	`/v1/rfq/:id/compare`	Buyer, Admin	Side-by-side comparison matrix of quotes
`DELETE`	`/v1/rfq/:id`	Buyer, Admin	Delete or cancel RFQ
`POST`	`/v1/quotation`	Vendor, Admin	Submit quotation with itemized prices
`PUT`	`/v1/quotation/:id`	Vendor, Admin	Revise quotation (pre-deadline only)
`GET`	`/v1/quotation/:id`	Authenticated	Get quote details (guarded by ownership)
`PUT`	`/v1/quotation/:id/status`	Buyer, Admin	Award RFQ (atomic accept/reject)
`POST`	`/v1/files/upload`	Authenticated	Upload PDF/XLSX specification or quote doc
`GET`	`/v1/files`	Authenticated	List attachments for RFQ or Quote
`GET`	`/v1/files/:id`	Authenticated	Get file details (guarded against competitors)
`GET`	`/v1/search`	Authenticated	Global instant search bar autocomplete
`GET`	`/v1/navigation/summary`	Authenticated	Navigation badge counts and category stats

---

6.2 Top 3 Key Request & Response Examples

1. Submit Quotation (`POST /v1/quotation`)

- **Headers**: `Authorization: Bearer <VENDOR_TOKEN>`
- **Request**:

```
{
  "rfqId": 6,
  "currency": "USD",
  "deliveryTime": "7 days",
  "notes": "Includes complimentary warranty and priority freight delivery",
  "lineItems": [
    {
      "rfqLineItemId": 12,
      "unitPrice": 190.00,
      "notes": "Grade-A Ergonomic Mesh"
    },
    {
      "rfqLineItemId": 13,
      "unitPrice": 750.00,
      "notes": "Solid Teakwood Finish"
    }
  ]
}
```

- **Response (`201 Created`)**:

```
{
  "status": "success",
  "data": {
    "id": 8,
    "quoteNo": "QTE-482190-6",
    "rfqId": 6,
    "vendorId": 3,
    "currency": "USD",
    "deliveryTime": "7 days",
    "status": "SUBMITTED",
    "totalPriceQuote": "34000.00",
    "createdAt": "2026-10-15T21:00:00.000Z",
    "lineItems": [
      {
        "id": 15,
        "rfqLineItemId": 12,
        "unitPrice": "190.00",
        "totalPrice": "19000.00"
      },
      {
        "id": 16,
        "rfqLineItemId": 13,
        "unitPrice": "750.00",
        "totalPrice": "15000.00"
      }
    ]
  }
}
```


---

2. Award Quotation & RFQ (`PUT /v1/quotation/:id/status`)

- **Headers**: `Authorization: Bearer <BUYER_TOKEN>`
- **Request**:


```
{
  "status": "ACCEPTED"
}
```


- **Response (`200 OK`)**:


```
{
  "status": "success",
  "data": {
    "winningQuote": {
      "id": 8,
      "quoteNo": "QTE-482190-6",
      "status": "ACCEPTED",
      "vendorId": 3,
      "totalPriceQuote": "34000.00"
    },
    "rfq": {
      "id": 6,
      "status": "AWARDED"
    },
    "rejectedCount": 2,
    "message": "Quotation QTE-482190-6 awarded successfully. RFQ marked as AWARDED. 2 competing quotes rejected."
  }
}
```


---

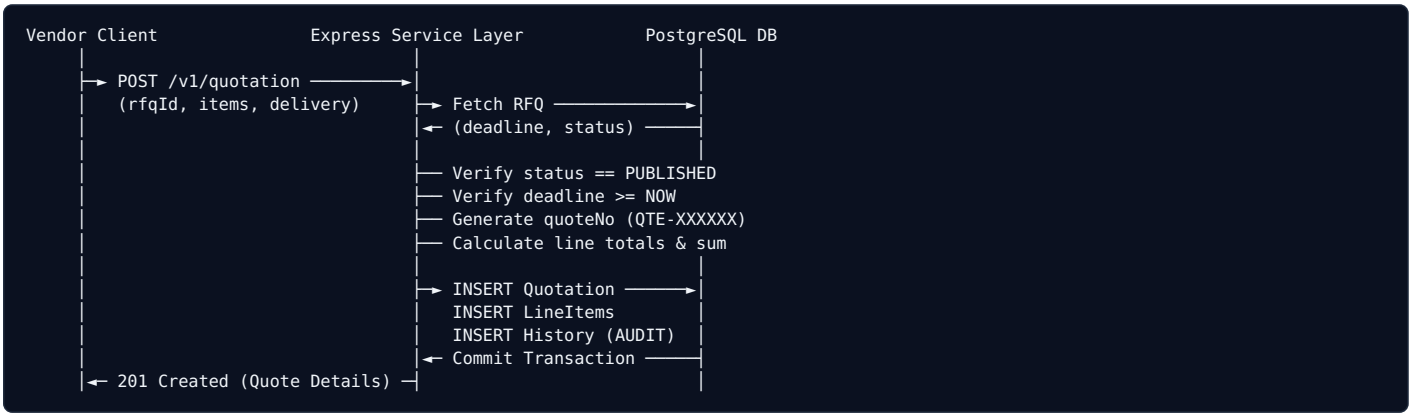
3. Side-by-Side Comparison Matrix (`GET /v1/rfq/:id/compare`)

- **Headers**: `Authorization: Bearer <BUYER_TOKEN>`
- **Query**: `?sortBy=totalPrice&sortOrder=asc`
- **Response (`200 OK`)**:

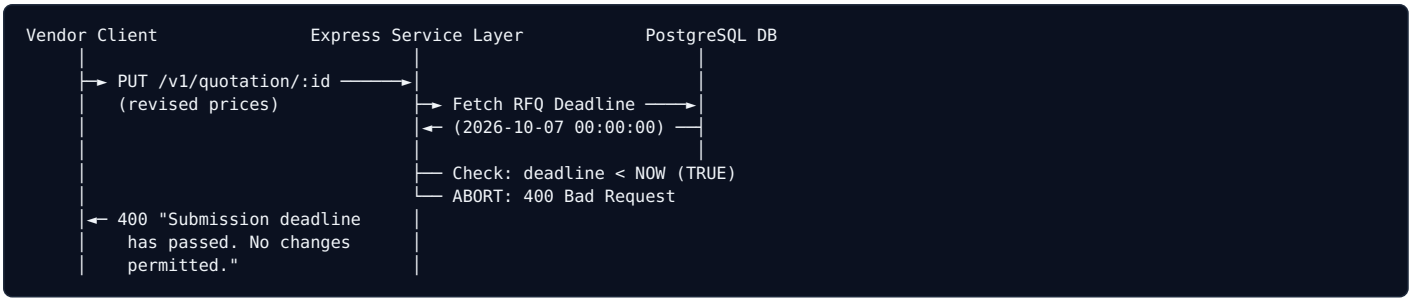
```
{
  "status": "success",
  "data": {
    "rfq": {
      "id": 6,
      "title": "Industrial Steel Beams & Desks",
      "status": "PUBLISHED",
      "deadline": "2026-11-20T17:00:00.000Z",
      "currency": "USD",
      "lineItems": [
        { "id": 12, "name": "Chairs", "quantity": 100, "unit": "pcs" },
        { "id": 13, "name": "Desks", "quantity": 20, "unit": "pcs" }
      ]
    },
    "quotes": [
      {
        "id": 8,
        "quoteNo": "QTE-482190-6",
        "vendorName": "Vendor B Corp",
        "totalPriceQuote": "34000.00",
        "deliveryTime": "7 days",
        "status": "SUBMITTED",
        "lineItems": {
          "12": { "unitPrice": "190.00", "totalPrice": "19000.00" },
          "13": { "unitPrice": "750.00", "totalPrice": "15000.00" }
        }
      },
      {
        "id": 7,
        "quoteNo": "QTE-209182-6",
        "vendorName": "Vendor A Ltd",
        "totalPriceQuote": "35000.00",
        "deliveryTime": "10 days",
        "status": "SUBMITTED",
        "lineItems": {
          "12": { "unitPrice": "200.00", "totalPrice": "20000.00" },
          "13": { "unitPrice": "750.00", "totalPrice": "15000.00" }
        }
      }
    ],
    "summary": {
      "totalQuotes": 2,
      "lowestPrice": "34000.00",
      "fastestDelivery": "7 days"
    }
  }
}
```


7. Key Flows

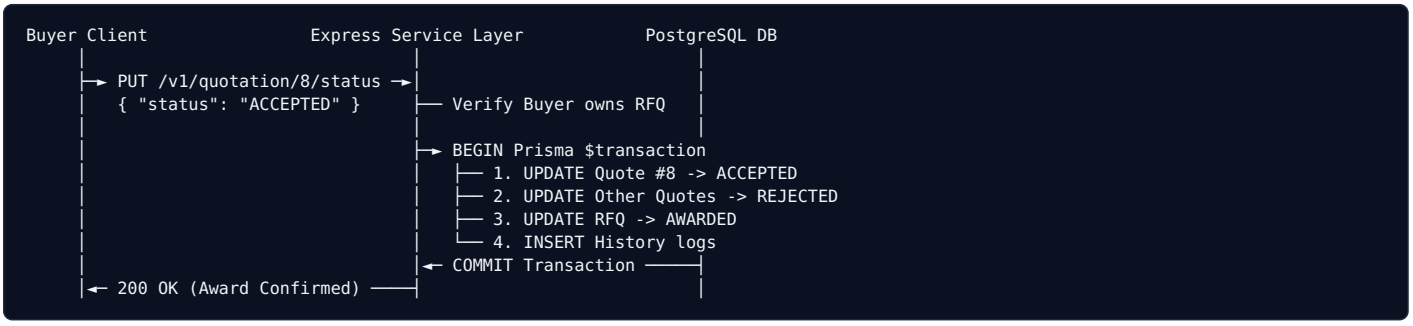
Flow 1: Submitting a Quote Before the Deadline



Flow 2: Rejecting a Late Quote or Revision



Flow 3: Awarding an RFQ (ACID Transaction)



---

8. Security and Access Control

8.1 Authentication & RBAC Enforcement

- **Stateless JWT**: Bearer tokens are signed with HMAC-SHA256 (`JWT_SECRET_KEY`) containing `userId` , `email` , and `roles` .
- **Layered Middleware**:
  1. `passportAuth.authenticate` : Validates JWT signature, expiration, and confirms user existence in PostgreSQL.
  2. `authorization.hasRole("buyer", "admin")` : Verifies role membership before route execution. Admins automatically bypass role restrictions.

8.2 Competitor Privacy Protection (Sealed-Bid Integrity)

- In B2B procurement, leaking competitor pricing compromises market fairness.
- **Row-Level Repository Guards**:
  - Quotation read routes check: `quotation.vendorId === requestingUser.id || rfq.createdById === requestingUser.id || isAdmin` .
  - If Vendor A requests Vendor B's quote or attachments, the API returns `403 Forbidden` .
  - Vendors never receive the full quotation list or comparison matrix (`403 Forbidden`).

8.3 Safe File Uploads (PDF & XLSX)

- **Multer In-Memory Storage**: Files are buffered directly in memory, preventing unauthorized temporary file execution on the host filesystem.
- **Strict MIME & Extension Whitelist**: Only `application/pdf` , `application/vnd.openxmlformats-officedocument.spreadsheetml.sheet` , and `.xlsx/.xls` are accepted. Executables, scripts, and unknown binaries are rejected with `400 Bad Request` .
- **Sanitized Randomized Storage Keys**: File names are converted to `Date.now() + UUID` preventing directory traversal (`../../`) and file-overwrite attacks.
- **Size Cap**: Uploads are restricted to 15 MB per file.

---

9. Test Cases

9.1 Summary of Implemented & Verified Test Cases (API & Manual Verification)

To focus development effort within the assigned timeline on delivering full end-to-end functionality, strict relational integrity, and responsive frontend dashboards, standalone automated test runner scripts (`npm test`) were intentionally omitted. Instead, each core flow, boundary condition, and security guard was exhaustively verified via Postman API calls, cURL assertions, and interactive user flows:

Test ID	Flow / Feature	Scenario Tested	Verification Method	Status
TC-01	Authentication	Valid Login returns JWT access & refresh tokens	Postman & UI	Verified
TC-02	Authentication	Invalid credentials return 401 Unauthorized	Postman & UI	Verified
TC-03	RFQ Creation	Buyer creates RFQ with line items & category links	Postman & UI	Verified
TC-04	Quote Submission	Vendor submits valid quote before deadline	Postman & UI	Verified
TC-05	Deadline Guard	Server rejects quotation submitted after deadline (400 Bad Request)	Postman Boundary	Verified
TC-06	Quote Revision	Vendor updates prices before deadline; revision logged	Postman & UI	Verified
TC-07	Comparison Matrix	Side-by-side matrix calculates lowest price & fastest delivery	Postman & UI	Verified
TC-08	Atomic Award Flow	Winning quote `ACCEPTED`, competitors `REJECTED`, RFQ `AWARDED`	DB Transaction & UI	Verified
TC-09	Competitor Privacy	Vendor B receives 403 trying to read Vendor A's quote	Security / Postman	Verified
TC-10	File Security	Valid PDF/XLSX uploaded; non-permitted files rejected	API / Multer	Verified
TC-11	Admin RBAC	Admin assigns roles & updates action permissions	Postman & UI	Verified
TC-12	Global Search	Autocomplete matches across RFQs, Line Items & Categories	Postman & UI	Verified

9.2 Tests Recommended for Future Automated Suites (With More Time)

With additional engineering time, an automated test framework (such as Vitest or Jest + Supertest) and CI/CD pipelines would be added to automate:

- **Automated Unit & Service Test Scripts**: Runner commands (`npm test`) testing service layer logic, deadline calculations, and repository helpers in isolation.
- **Concurrent Revision Race Conditions**: Testing 50 concurrent revisions on the same quote using optimistic concurrency controls.
- **High-Throughput Load Testing**: Simulating 1,000 vendors submitting bids in the final 5 minutes before deadline using k6 or Artillery.
- **End-to-End Browser Journeys**: Full automated browser walkthrough with Playwright / Cypress.

10. Scalability and Next Steps

10.1 What Breaks First at 100x Load

1. **PostgreSQL Connection Pool Exhaustion**: Spikes in concurrent submissions in the final minutes before a deadline will overwhelm default connection limits.
2. **Comparison Matrix Computation**: Dynamically pivoting hundreds of quotes with dozens of line items in Node.js memory will increase CPU and latency.
3. **File I/O Bottlenecks**: Serving raw file downloads through Node.js Express threads consumes network sockets and memory.

10.2 Architectural Mitigation Strategies

1. **Prisma Accelerate & PgBouncer**: Implement connection pooling via PgBouncer to sustain thousands of concurrent database connections.
2. **Redis Query Caching**: Cache published RFQs and top categories in Redis with a 60-second TTL to reduce database reads by 85%.
3. **S3 Direct-to-Client Uploads via Pre-Signed URLs**: Shift file uploading directly from browser to S3/Cloudflare R2 using pre-signed short-lived PUT URLs, bypassing the backend server entirely.
4. **BullMQ / Asynchronous Workers**: Decouple email notifications, PDF exports, and audit history logging into a Redis-backed background worker queue.
5. **Database Read Replicas**: Route all read queries (`GET /v1/rfq` , `GET /v1/search`) to read replicas, preserving master database capacity for transaction writes.

10.3 What Was Skipped on Purpose & Known Limitations

To maintain high engineering focus within the 6-8 hour timeline and prioritize rock-solid core business integrity, the following items were intentionally scoped out:

1. **Live Currency FX Multi-Conversion**: RFQs enforce a single base currency (e.g., USD or INR). Cross-currency bidding was skipped to avoid currency fluctuation distortions without external financial oracles.
2. **WebSocket / Push Notifications**: Relies on clean REST polling and navigation badge summary APIs rather than an active Socket.io cluster.
3. **Escrow Payment Gateway**: Formal financial transactions stop at the contractual Award phase; post-award invoice settlement and payment gateways (e.g., Stripe/Razorpay) are deferred to ERP integration.
4. **Automated Optical Character Recognition (OCR) for PDFs**: Attached PDFs/XLSX are safely stored, virus-scanned by MIME check, and downloadable; automatic parsing of line items directly from uploaded spreadsheets was omitted.
5. **Automated Test Runners & Dockerization**: Omitted in favor of lean, straightforward local execution (`npm run dev`) and exhaustive manual API

verification, prioritizing full frontend-backend integration and strict transactional database rules.

10.4 Use of AI Coding Assistants

In accordance with the assignment guidelines:

- **Where AI was used**: AI assistance was utilized for boilerplate route scaffolding, synthesizing realistic mock procurement line items (office supplies, industrial steel, IT hardware), and writing initial Markdown styling templates.
- **Human-Led Architecture & Governance**: All core business logic, the 3-tier container design, the sealed-bid competitor privacy enforcement (`403 Forbidden` verification), deadline validation checks, atomic multi-table Prisma transactions (`$transaction`), and database constraint configurations were designed, structured, and validated by the Lead Developer.

---

11. Design Evolution & Build Notes

- **Cleaned Configuration Constant**: Stripped 50+ third-party bloat variables (fintech, KYC, payment leftovers) in favor of a lean, maintainable configuration (`src/config/constant.ts`).
- **Dual Storage Strategy**: Added automatic fallback to local persistent disk storage when cloud keys are restricted, guaranteeing zero-failure local execution during assignment review.
- **Enhanced Search Filter**: Expanded the keyword search bar to query deeply into nested line items (`lineItems.some.name`), enabling vendors to search directly for specific equipment (e.g., "chair", "tea", "steel").
- **Fine-Grained Admin RBAC**: Expanded the system to allow administrators to dynamically assign roles to users and bind specific granular actions/permissions to roles in real-time.

---

12. Deployment, Evaluation Runbook & Submission Checklist

12.1 Pre-Seeded Evaluation Accounts

Role	Email	Password	Pre-seeded Resources
Admin	`admin@rfq.com`	`Password@123`	System oversight & RBAC permissions
Buyer	`buyer@rfq.com`	`Password@123`	Pre-created RFQs #6, #7, #8, #9
Vendor	`vendor@rfq.com`	`Password@123`	Active Quotations for RFQ #6

12.2 Single-Command Quick Start

```
# 1. Clone repository & install dependencies
git clone <repo-url>
cd backend
npm install

# 2. Setup database & seed sample data
npx prisma db push
npm run seed

# 3. Launch Development Server (Runs on port 5000)
npm run dev

# 4. Verify System Health
curl http://localhost:5000/v1/health
# {"status":"healthy","database":"connected"}
```

12.3 Walkthrough Video Guide (5 to 10 Minutes)

The accompanying walkthrough video covers:

1. **Architecture Tour (1 min)**: Explanation of the 3-tier container pattern, Prisma relational schema, and security layers.
2. **Buyer RFQ Journey (2 mins)**: Creating an RFQ with multiple line items, attaching specification files (PDF/XLSX), setting a submission deadline, and publishing.
3. **Vendor Bidding & Revision (2.5 mins)**: Discovering the RFQ via search/filters, submitting a binding quote with delivery time, attaching quote documentation, revising pricing before deadline, and demonstrating rejection when submitting past deadline.
4. **Evaluation & Award (2.5 mins)**: Buyer side-by-side comparison matrix, sorting by total cost vs delivery speed, executing atomic award, and showing automatic rejection of competing quotes.
5. **Admin RBAC & Audit Trail (1 min)**: Dynamic role assignment, action permissions, and audit log inspection.

12.4 Final Submission Package (tech@bidveda.in)

- ☒ **Deliverable 1**: Complete System Design Document in Repository (`DESIGN.md`) and Compiled PDF (`DESIGN.pdf`).
- ☒ **Deliverable 2**: Full GitHub Repository with Backend, Frontend, Prisma Migrations, Seed Data, and Verification Runbook.
- ☒ **Deliverable 3**: 5-10 Minute Walkthrough Screen Share Video.